

MODEL TELLS YOU WHERE TO MERGE: ADAPTIVE KV CACHE MERGING FOR LLMs ON LONG-CONTEXT TASKS

Zheng Wang¹, Boxiao Jin¹, Zhongzhi Yu¹, Minjia Zhang²

¹Georgia Institute of Technology, ²University of Illinois Urbana-Champaign
{zwang3478, bjin60, zyu401}@gatech.edu, {minjiaz}@illinois.edu

ABSTRACT

Large Language Models (LLMs) have attracted remarkable attention due to their unprecedented performance across a wide range of tasks. However, how to efficiently serve LLMs has become a pressing issue because of their huge computational cost in their autoregressive generation process. To mitigate computational costs, LLMs often employ the *KV Cache* technique to improve the generation speed. While improving the computational efficiency, the storage requirements of the KV cache are substantial, particularly in long-context scenarios, leading to significant memory consumption. Existing KV cache eviction methods often degrade the performance of LLMs in long-context scenarios due to the information loss introduced by eviction. In this paper, we propose a novel KV cache merging approach, called *KVMerger*, to achieve adaptive KV cache compression for long-context tasks without significant performance degradation under constrained memory budgets. Our approach is inspired by the intriguing observation that key states exhibit high similarity at the token level within a single sequence. To facilitate merging, we develop an effective yet straightforward merging set identification algorithm to identify suitable KV states for merging. Our merging set identification algorithm stimulates the second observation that KV cache sparsity, from similarity perspective, is independent of the dataset and remains persistent at the model level. Subsequently, we propose a Gaussian kernel weighted merging algorithm to selectively merge all states within each merging set. We conduct extensive experiments to demonstrate the effectiveness of *KVMerger* for long-context tasks under constrained memory budgets, applying it to models including Llama2-7B/13B-chat and Mistral-7B-instruct. Using the LongBench and Zero-Scroll benchmarks, we compare our method with other KV cache compression techniques, including H2O and CaM, showing that our method achieves superior performance across tasks with both 50% and 35% KV cache budgets.

1 INTRODUCTION

Large Language Models (LLMs) have demonstrated exceptional performance across a variety of applications, particularly excelling in long-context scenarios that are increasingly relevant in everyday life. Recent state-of-the-art LLMs have been meticulously developed to scale up to handle long-context tasks, such as OpenAI’s ChatGPT (OpenAI, 2023), Anthropic’s Claude (Anthropic, 2023), Meta’s LLaMA-3 (Touvron et al., 2023a) (Touvron et al., 2023b), Mistral (Jiang et al., 2023), and Google’s Gemini-pro-1.5 that supports a staggering 1M token context length (Gemini Team, 2024). However, as LLMs process larger volumes of data over extended contexts, *KV cache* starts to pose a substantial obstacle to LLM’s performance and scalability. KV cache stores the key and value states (KV) derived from the attention calculation of previously processed tokens and reuses those states in the autoregressive generation process. As LLMs continue to grow in size and capabilities, supporting long-context starts to eat up memory. For example, a 175-billion parameter GPT-3 model, with a batch size of 64 and a sequence length of 4,096 tokens (including both prefilled and generated tokens), necessitates approximately 1,208 GB of GPU memory (Liu et al., 2024), which exceeds the

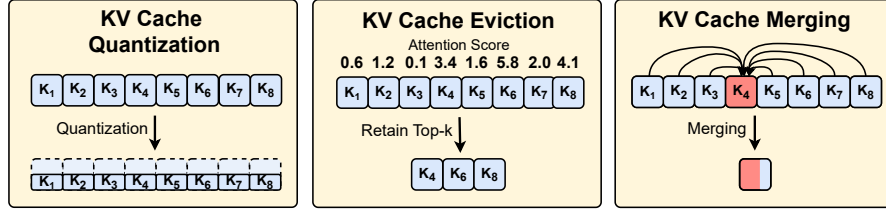


Figure 1: Three categories of KV cache compression techniques: KV cache quantization (left), KV cache eviction (middle), and KV cache merging (right). For the illustration of KV cache eviction, we use aggregated attention scores as the eviction signal, and k is set to 3; for KV cache merging, we illustrate many-to-one merging. The key state in red represents the state which incorporates the information of other remaining states. Value states are processed in the same way as key states.

memory capacity of most advanced GPUs. Therefore, the need for compressing KV cache while maintaining LLM generation quality, especially for long-context tasks, becomes essential.

Current efforts for KV cache compression can be broadly categorized into three types: quantization, eviction, and merging, as illustrated in Figure 1. Quantization replaces floating point KV states (e.g., FP16) with low-bit representations to decrease memory usage while striving to maintain the overall performance of LLMs. Recent advancements, such as Coupled Quantization (Zhang et al., 2024b) and KIVI (Zirui Liu et al., 2023), have demonstrated that KV cache can be quantized to 1-bit or 2-bit precision while preserving performance. In contrast, KV cache eviction methods selectively remove unimportant tokens from the cache based on certain signals from the model, thereby reducing the memory footprint by limiting the number of key and value states in the KV cache (Xiao et al., 2024; Liu et al., 2023b; Zhang et al., 2023; Ge et al., 2024). For instance, Scissorhands (Liu et al., 2023b) keeps a fixed KV size budget and relies on the Persistence of Importance hypothesis to evict key and value states for non-important tokens. Similarly, H2O (Zhang et al., 2023) utilizes aggregated attention scores to determine so called “heavy hitters”, which are a subset of important tokens to keep in the KV cache. While eviction-based methods have demonstrated promising results on short context tasks with simple perplexity metrics, a significant drawback of eviction methods is their potential to accidentally and permanently remove important tokens, leading to context damage and adversely affecting their effectiveness in long-context tasks that heavily rely on context information. On a separate line of research, KV cache merging has been proposed as a complementary method of eviction (Zhang et al.; Wan et al., 2024; Liu et al., 2024; Yu et al., 2024a).

Unlike eviction-based methods, the KV cache merging technique does not strictly remove key and value states. Instead, it involves merging states that are otherwise to be dropped by eviction method into single token state. By amalgamating states rather than outright evicting them, this method ensures that essential information not captured by the attention scores is retained, thereby enhancing the model’s ability to maintain performance and accuracy in long-context tasks with compressed KV cache. It is noteworthy that, although token merging is well-established in computer vision (CV) (Zeng et al., 2022) (Bolya et al., 2023) (Kim et al., 2023) (Zhang et al., 2024a), the application of key and value states merging in LLMs has not been extensively explored due to several significant challenges. Specifically, *the high dimensionality and sparsity of KV cache make it difficult to accurately identify sets of states that can be merged without losing critical information*. Additionally, *developing appropriate merging algorithm without introducing the loss of essential information in long context presents another major challenge*. Effective merging techniques must strike a delicate balance between reducing memory usage and preserving the semantic integrity of the contexts.

To address the aforementioned challenges associated with KV cache merging, we propose an effective KV cache merging method for accelerating autoregressive LLMs, especially for improving its performance in long-context tasks. We start by introducing an intriguing observation: *key states exhibit high cosine similarity at the token level within a single sequence across different attention heads and model layers*. We investigate the root cause of why such phenomenon appears, and our observation also opens opportunities for effective merging of key and value states based on their cosine similarity. Subsequently, we formulate the KV cache merging as a constrained clustering problem, and we introduce a strong baseline for this problem, where we use an effective merging set identification method for KV cache merging, which results in a layer-wise KV cache compression together with a simple weighted merging algorithm. Based on the proposed merging set identification method, we define KV cache sparsity from the perspective of states similarity. Our finding indi-

cates that KV cache sparsity is independent of the dataset and remains persistent at the model level. Building on top of this, we propose a Gaussian kernel weighted merging algorithm to merge states within each identified merging set. We compare our proposed method with existing KV cache eviction method H2O and value states merging method CaM. The results demonstrate that our method achieves a better performance on these two benchmarks with both 50% and 35% KV cache budgets, surpassing existing KV cache eviction methods. Our contributions can be summarized as follows:

- As one of the pioneering researches concerning KV cache merging for LLMs, we developed *KVMerger*, an effective KV cache merging algorithm especially designed for long-context tasks, including merging set identification and Gaussian kernel weighted merging function.
- We introduce an intriguing observation that key states share a high similarity at the token level within a single sequence, as an important complementary to the previous observations concerning high query states similarity (Dai et al., 2024) and intra-layer KV cache similarity (Liu et al., 2024). We also investigate the root cause of why such phenomenon appears.
- Our proposed *KVMerger* outperforms the previous KV Cache eviction algorithms on long-context tasks across various models under both 50% and 35% KV cache budgets, introducing a great memory reduction compared to full KV cache.

2 RELATED WORK AND PROBLEM FORMULATION

2.1 KV CACHE QUANTIZATION

Quantization methods involve converting high-precision numerical values of key and value states into lower-precision formats, thereby decreasing the storage requirements within the cache (Hooper et al., 2024; Sheng et al., 2023; Liu et al., 2023a; Zhang et al., 2024c). Due to the presence of outliers in key and value states, recent works such as KIVI (Zirui Liu et al., 2023) and Gear (Kang et al., 2024) employ fine-grained group-wise quantization, which quantize small channel groups within each token. MiKV (Yang et al., 2024) addresses the information loss introduced by KV cache eviction methods by preserving those KVs in lower precision rather than directly dropping them. Zip-Cache (He et al., 2024) proposes an efficient channel-separable quantization scheme, disentangling the channel and token dimensions without excessive memory overhead. Different from quantized KV cache optimizations, this work studies compression of KV cache via token merging, which is complementary to quantization and can lead to better improvements when combined together.

2.2 KV CACHE EVICTION

KV cache eviction methods focus on retaining those important key-value pairs and discard those unimportant ones permanently. One of the common selection policies of key-value pairs is to exploit signals from the attention mechanism of LLMs to select important tokens. For example, H2O (Zhang et al., 2023), Scissorhands (Liu et al., 2023b), and RoCo (Ren & Zhu, 2024) compress KV cache by maintaining a small set of KV states whose corresponding tokens are determined by the ranking of attention scores. StreamingLLM (Xiao et al., 2024) finds that keeping the initial tokens, called attention sink, together with the recent window tokens is pivotal to maintain LLM’s performance. More recently, Ge et al. (2024) and Yu et al. (2024b) find that attention sinks also occurs in the middle of the sentences, and Ge et al. (2024) introduces FastGen which can choose the most appropriate compression strategy for each heads with different attention distribution patterns. While demonstrating promising results, existing eviction methods are often evaluated on simple and widely questioned metrics, e.g., perplexity, which may fail to capture LLM’s capabilities in understanding long contexts. In contrast, we specifically look into KV compression under more challenging long-context understanding tasks.

2.3 KV CACHE MERGING

Instead of permanently discarding key and value states, KV cache merging offers a promising direction for KV cache compression while maintaining the performance of LLMs, particularly for long-context tasks such as Retrieval-Augmented Generation (RAG). MiniCache (Liu et al., 2024) finds that KV states of some consecutive layers have high similarity and proposes an effective intra-layer KV cache merging and restoration algorithms to reduce memory usage by KV cache. CaM

(Zhang et al.) adaptively merges to-be-evicted value states into the remaining conserved value states, resulting in minimal output perturbation due to the merging operation. Similarly, D2O Wan et al. (2024) selectively merges both value and key states to be evicted with those to be conserved using an Exponential Moving Average (EMA) threshold, and uses weighted merging based on cosine similarity. However, these methods are highly dependent on previous eviction methods, and how to identify effective merging set for KV cache and define effective merging method still remains unclear for KV cache. This paper is the first one to consider KV cache problem independently and propose simple yet effective solutions.

2.4 PROBLEM FORMULATION

Formally, we study the performance impact of LLMs after compressing (without fine-tuning) their KV cache. For a decoder only pre-trained LLM f , we denote its key states and value states as $\mathcal{K} \in \mathbb{R}^{n \times d}$ and $\mathcal{V} \in \mathbb{R}^{n \times d}$, respectively. Let Q_t denote the query state at time step t , and $Q_t \in \mathbb{R}^{1 \times d}$. Then, the output $\mathcal{O}_t$ for each attention head at a certain layer of f can be formulated as:

$$\mathcal{O}_t = \mathcal{A}_t \mathcal{V}, \mathcal{A}_t = \text{softmax} \left(\frac{Q_t \mathcal{K}^T}{\sqrt{d_k}} \right) \quad (1)$$

KV Cache Merging Algorithm. Our primary objective is to develop an efficient many-to-one merging algorithm M for KV cache, which should generate merged key states $M(\mathcal{K}) = \bigcup_{i=1}^d F(\mathcal{S}_{ki})$ and merged value states $M(\mathcal{V}) = \bigcup_{i=1}^d F(\mathcal{S}_{vi})$, where $\mathcal{S}_{ki}$ and $\mathcal{S}_{vi}$ represent the sub-merging sets for key states and value states, respectively. F is the merging function which maps the states in each merging set to a single state. Note that $\mathcal{K} = \bigcup_{i=1}^d \mathcal{S}_{ki}$ and $\mathcal{V} = \bigcup_{i=1}^d \mathcal{S}_{vi}$.

Definition 2.1 (KV Cache Merging Problem, informal). *Let $\mathcal{O}_t$ represent the original output of each attention head at a certain layer of f , and let $\mathcal{O}_t^*$ represent the output after merging. M must satisfy the following optimization criterion:*

$$M = \arg \min_M \frac{|M(\mathcal{K})|}{|\mathcal{K}|}, \quad (2)$$

subject to $|\mathcal{O}_t - \mathcal{O}_t^| \leq \epsilon$, where ϵ is an acceptable small positive value, ensuring that the degradation in performance is negligible and within acceptable bounds. M also has the following properties:*

- $|M(\mathcal{K})| / |\mathcal{K}| \leq 1, |M(\mathcal{V})| / |\mathcal{V}| \leq 1$
- $|M(\mathcal{K})| / |\mathcal{K}| = |M(\mathcal{V})| / |\mathcal{V}|$ (make sure key and value states have the same compression ratio)

In our study, the merging algorithm M consists of two parts: (i) identifying sub KV cache sets with a policy I and (ii) determining the suitable merging function F for each set, where F ensures the states within each set are merged to a single state, such that for a task T , the resulting compressed KV cache leads to a performance drop no more than ϵ (compression tolerance threshold).

KV Cache Merging Sets Identification Policy. We define the identification policy I as:

- $|\mathcal{K}| = |\mathcal{K}_c| + |\mathcal{K}_m|, |\mathcal{V}| = |\mathcal{V}_c| + |\mathcal{V}_m|$
- $|\mathcal{K}_c| = |\mathcal{V}_c|, |\mathcal{K}_m| = |\mathcal{V}_m|$ (make sure key states and value states come in pair)

where $\mathcal{K}_c$ and $\mathcal{K}_m$ represent the subsets of key states to be conserved and merged, respectively, and $\mathcal{V}_c$ and $\mathcal{V}_m$ represent the subsets of value states to be conserved and merged, respectively. The above definition is a general formulation. For example, when $|\mathcal{K}_c|$ and $|\mathcal{V}_c|$ are zero, all key and value states are merged, resulting in a full cache without any states eviction.

KV Cache Merging Function. We define the merging function F such that

$$F : \{S_i\}_{i=1}^d \rightarrow \{s_i^*\}_{i=1}^d, \text{ where } F(S_i) = s_i^*, \quad i = 1, 2, \dots, d$$

where s_i^* is the merged new state for each sub merging set.

3 OBSERVATIONS

In this section, we present two key observations illustrating that KV cache sparsity is universal for long-context tasks when viewed from the perspective of state similarity. These observations form the basis for our development of the adaptive KV cache merging algorithm, *KVMerger*.

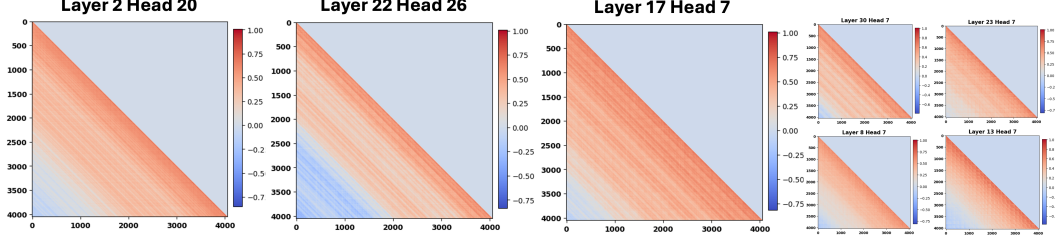


Figure 2: Visualization of the cosine similarity map of key states at the token-wise level produced by running the inference process on the Llama2-7b-chat model by randomly sampling data from the SynthWiki dataset. Observations include: (1) Key states share strong similarity within one sequence across different layers and heads; (2) The similarity between key states has the property of locality, i.e., adjacent tokens exhibit higher similarity.

3.1 KV CACHE SIMILARITY

Previous literature have analyzed the possibility of reducing KV cache size via exploiting attention sparsity, i.e., identifying important tokens via their attention scores (Zhang et al., 2023) (Liu et al., 2023b). However, *attention-score-driven* approaches are biased (He et al., 2024) because critical tokens often vary a lot across different queries (Tang et al., 2024), where relying on attention-scores alone can lead to context damage. Instead of relying on attention scores, we investigate whether merging token states can preserve critical context details. Inspired by Dai et al. (2024), which reveals the phenomenon that query states share significant similarity at the token level in LLMs, we observe for the first time that *key states also exhibit very high similarity at the token level within single sequence*. We will first demonstrate the generalization of this token level similarity in key states and then analyze the potential reasons behind this intriguing observation.

Observation: key states exhibit high, localized token-level similarity. We conduct the inference process on the Llama2-7b-chat model by randomly sampling data from the SynthWiki dataset (Peysakhovich & Lerer, 2023) with average sequence length being about 4000. Then, we visualize the cosine similarity of key states at the token-wise level within a sequence using the following equation:

$$\text{similarity}(k_i, k_j) = \frac{k_i k_j^T}{\|k_i\| \cdot \|k_j\|}, \quad 1 \leq i, j \leq T, \quad (3)$$

where T is the total length of the sequence. k_i represents the i -th key state, and k_j represents the j -th key state. The visualization results are illustrated in Figure 2. We can observe that the similarity maps illustrate a clear oblique color segmentation, and the closer it is to the diagonal, the more intense the color becomes, indicating that key states exhibit a strong localized similarity as query states do (Dai et al., 2024). Specifically, key states share extremely high similarity values with its adjacent tokens, which is greater than 90% for some tokens as Figure 3(a) shows. Moreover, we also observe from Figure 3(a) that the local similarity between one value states and the other consecutive key states shows different fluctuations for different attention heads. We also examine the cosine similarity of value states but do not observe the local similarity property. One interesting question arises: *why do such localized token similarity exhibit in key states, while value states do not?*

Analysis. Recent advancements in large language models (LLMs), including Llama2, Mistral, and Llama3, have showcased significant performance improvements by employing Rotary Position Embedding (RoPE) (Su et al., 2023). RoPE integrates positional information into token embeddings through a rotational transformation based on positional indices. This process utilizes sinusoidal functions, specifically cosine and sine components, to encode positions. By rotating the embeddings in a multi-dimensional space, RoPE effectively captures the relative positions and order of tokens within a sequence. If we denote two adjacent input tokens as $x_m, x_n \in \mathbb{R}^d$ where n and m are two random integers, then in RoPE, the position information of each token is incorporated via the following equations:

$$k_{m, [2j:2j+1]} = W_k x_m e^{im\theta_j}, \quad k_{n, [2j:2j+1]} = W_k x_n e^{in\theta_j}, \quad \theta_j = b^{-\frac{2j}{d}}, \quad (4)$$

where W_k is the matrix for key projection, b is called as the rotary base, which is set to 10000 by default (Su et al., 2023).

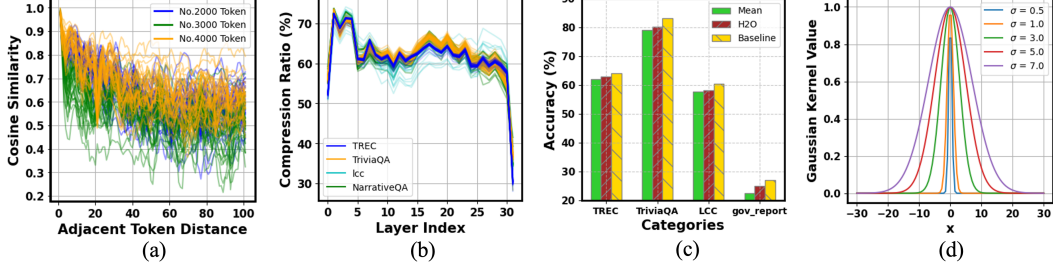


Figure 3: (a): The cosine similarity changes between the current token and its adjacent tokens across distinct attention heads and layers. We show the above changes for tokens with indices being 2000, 3000, and 4000. (b) The layer-wise compression ratios obtained by our proposed merging set identification algorithm for different samples and different tasks. (c) The comparison of long-context performance between H2O and average weighted merging with our proposed merging set identification algorithm. (d) The illustration of Gaussian kernel function with different values of σ .

Lemma 3.1 (Informal). Consider two vectors $\mathbf{k}_m, \mathbf{k}_n \in \mathbb{R}^{1 \times d}$. If their cosine similarity is 1, then the cosine similarity of any 1×2 vectors, $\mathbf{k}_{m,j} = [k_{m,2j}, k_{m,2j+1}]^T$ and $\mathbf{k}_{n,j} = [k_{n,2j}, k_{n,2j+1}]^T$, formed by the $2j$ -th and $(2j+1)$ -th elements of $\mathbf{k}_m$ and $\mathbf{k}_n$, $0 \leq j \leq (d-1)/2$, is also equal to 1.

Lemma 3.2 (Informal). Consider integer j such that $0 \leq j \leq \frac{d-1}{2}$. Define the vectors $\mathbf{k}_{m,j}$ and $\mathbf{k}_{n,j}$ as $\mathbf{k}_{m,j} = [k_{m,2j}, k_{m,2j+1}]^T$ and $\mathbf{k}_{n,j} = [k_{n,2j}, k_{n,2j+1}]^T$, and define the vectors $\mathbf{k}'_{m,j}$ and $\mathbf{k}'_{n,j}$ as $\mathbf{k}'_{m,j} = \mathbf{k}_{m,j}/e^{im\theta_j}$ and $\mathbf{k}'_{n,j} = \mathbf{k}_{n,j}/e^{in\theta_j}$. If similarity $(\mathbf{k}_{m,j}, \mathbf{k}_{n,j}) = 1$, we have:

$$\cos(m-n) < \frac{\langle \mathbf{k}'_{m,j}, \mathbf{k}'_{n,j} \rangle}{\|\mathbf{k}'_{m,j}\| \cdot \|\mathbf{k}'_{n,j}\|} \leq 1,$$

where $\langle \mathbf{k}'_{m,j}, \mathbf{k}'_{n,j} \rangle$ denotes the inner product of $\mathbf{k}'_{m,j}$ and $\mathbf{k}'_{n,j}$, and $\|\mathbf{k}'_{m,j}\|$ and $\|\mathbf{k}'_{n,j}\|$ denote the norms of $\mathbf{k}'_{m,j}$ and $\mathbf{k}'_{n,j}$, respectively.

The formal and complete proof of the above lemma is shown in appendix A. The conclusions of lemmas 3.1 and 3.2 are the necessary conditions of $\text{similarity}(\mathbf{k}_{m,j}, \mathbf{k}_{n,j}) = 1$. A cosine similarity of $\mathbf{k}'_{m,j}$ and $\mathbf{k}'_{n,j}$ falling beyond the range $(\cos(m-n), 1]$ will result in the failure of $\text{similarity}(\mathbf{k}_{m,j}, \mathbf{k}_{n,j}) = 1$. The analysis above clarifies why value states exhibit low similarity at the token level. Without the RoPE operation, value states are incapable of achieving rotation to comparable angles. Both empirical observations and theoretical analysis indicate that merging highly similar key states is approachable. This scheme is preferable to simply discarding key states, as it helps prevent potential information loss, particularly in long-context scenarios.

3.2 PERSISTENT KV CACHE SPARSITY

We have demonstrated that key states within a sequence exhibit significant similarity at the token level in pre-trained LLMs. Based on this, we progressively group consecutive key states of a given key state with similarity values exceeding a certain threshold. By applying this process from the last token to the first token, we obtain a set of groups, each containing consecutive key states with high similarity above the specified threshold. The obtained new key states set is defined as the merging set, meaning that the number of groups in the obtained set equals to the number of key states after merging. The above set identification algorithm is described in detail in Section 4.1.

Observation: The KV cache sparsity for different samples are persistent at the model level.

Figure 3(a) shows that the similarity distributions of different tokens vary across distinct attention heads and layers. The size of each subset of key states is governed by the similarity threshold defined. Lowering the threshold results in the inclusion of a larger number of key states within a single merging set, thereby leading to varied compression ratios across all attention heads and layers. To investigate the actual compression ratios achieved by the previous set identification algorithm, we conduct inference processes on the Llama2-7b-chat model. This involves randomly sampling 200 instances from the subset of LongBench (Bai et al., 2024) tasks and calculating the average compression ratio for each layer, as shown in Figure 3(b). We observe that the layer-wise compression

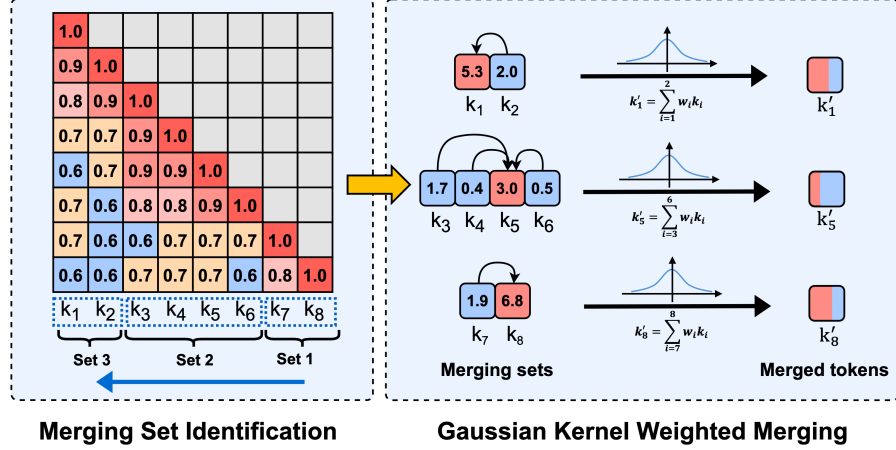


Figure 4: The whole framework of *KVMerger* is comprised of two major modules. The first module is to identify the merging set through our proposed algorithm in Section 4.1. Note that those key and value states which are most sensitive to merging are excluded. The toy similarity map is used to illustrate this process in the above Merging Set Identification part, and the threshold for cosine similarity is set to 0.8. The second module is to merge key and value states within each identified merging set via Gaussian kernel weighted merging as described in Section 4.2. For Gaussian kernel weighted merging illustration, the key state in red color represents the pivotal key state, where all the remaining key states should be weighted merged to that one. Note that values on key states in the above graph represent the aggregated attention scores.

ratios were highly consistent across different samples from the same task and even across different tasks. This intriguing finding suggests that the *kv cache sparsity, resulting from the high similarity exhibited by key states, is independent of the dataset and remains persistent at the model level.*

Insights The observed static KV cache sparsity suggests that it is possible to determine the layer-wise compression ratios by adjusting the cosine similarity threshold, thereby reducing the KV cache memory consumption. Additionally, Figure 3(b) shows that the first two layers and the last few layers have relatively small compression ratios. This observation aligns with previous research indicating that the attention score distributions are more uniform in the first two layers and last one layer of LLMs (Yu et al., 2024b) (Wan et al., 2024), suggesting that most key states are important and should be preserved to avoid introducing significant noise for those layers.

4 PROPOSED ADAPTIVE KV MERGING ALGORITHM

In this section, we propose *KVMerger*, an adaptive KV merging algorithm, for LLMs based on the above observations. The whole pipeline of *KVMerger* is depicted in Figure 4, from which we can see that the whole algorithm contains two major modules: merging set identification and Gaussian kernel weighted merging process. We first introduce the merging set identification algorithm in Section 4.1, which can be viewed as solving a constrained clustering problem. We propose a transformation of Agglomerative Hierarchical Clustering (AHC) algorithm to solve this. In Section 4.2, we delineate our proposed Gaussian kernel weighted merging algorithm, which is a many-to-one states merging method without introducing significant information loss.

4.1 GREEDY POLICY FOR MERGING SET IDENTIFICATION

One way to solve the merging set identification problem described in Section 2.4 is to view it as a variant of clustering problem, which we define below:

Definition 4.1 (Constrained Clustering Problem for KV Cache Merging, formal). *Given the original set of key states to be merged $\mathcal{K}_m = \{k_1, k_2, \dots, k_n\}$ from a certain attention head at a certain layer of f , where each $k_n \in \mathbb{R}^{1 \times d}$, and a similarity function $\delta : \mathcal{K}_m \times \mathcal{K}_m \rightarrow \mathbb{R}^{n \times n}$, partition $\mathcal{K}_m$ into d merging sets $\{\mathcal{S}_1, \mathcal{S}_2, \dots, \mathcal{S}_d\}$ such that the intra-cluster similarity is maximized and the inter-cluster similarity is minimized.*

- Each merging set $\mathcal{S}_i$ should satisfy: $\mathcal{S}_i \cap \mathcal{S}_j = \emptyset$ for $i \neq j$, and $\bigcup_{i=1}^d \mathcal{S}_i = \mathcal{K}_m$;

- $\forall \mathcal{S}_i, \exists j$ such that $\mathcal{S}_i = \{k_j, k_{j+1}, \dots, k_{j+|\mathcal{S}_i|-1}\}$;
- The objective function to be maximized can be expressed as:

$$\max_{\mathcal{S}_1, \mathcal{S}_2, \dots, \mathcal{S}_d} \left(\sum_{i=1}^d \sum_{k_a, k_b \in \mathcal{S}_i} \delta(k_a, k_b) - \sum_{(i,j), i \neq j} \sum_{k_a \in \mathcal{S}_i, k_b \in \mathcal{S}_j} \delta(k_a, k_b) \right)$$

The similarity function, δ , we used here is cosine similarity based on the observation in Section 3.1. In order to conserve the locality similarity property of key states, the merging set identification problem is a constrained clustering problem, meaning that all elements in one cluster should be consecutive in sequence, and we do not merge states with high similarity but far away from each other. Then, we propose a variant of Agglomerative Hierarchical Clustering (AHC) algorithm to find all merging sets shown as Algorithm 1.

Algorithm 1 Merging Set Identification

```

1: procedure AHC( $\mathcal{K}_m = \{k_1, \dots, k_T\}, \delta, \epsilon$ )
2:   Start with T clusters with one key state
3:   Compute the similarity matrix  $\Delta$  where
      $\Delta(i, j) = \delta(k_i, k_j)$ 
4:   for  $i = T \rightarrow 1$  do
5:     Group  $\{k_i\} \cup \{k_j \mid \delta(k_i, k_j) > \epsilon\}$ ,
       s.t.  $\|i - j\| = 1$ 
6:      $i = j$ 
7:   end for
8:   return The merging sets
9: end procedure

```

Algorithm 2 Merging Policy

```

1: procedure MERGE( $\mathcal{S}_k = \{k_1, \dots, k_n\}, A$ )
2:   Compute aggregated attn score for each
     state:  $\mathbf{a}_i = \sum_{i=1}^{|\mathcal{S}_k|} A[i, :]$ 
3:   Find pivotal state:  $p = \operatorname{argmax}_{i \in \mathcal{S}_k} (\mathbf{a}_i)$ 
4:   for  $j = 1 \rightarrow n$  do
5:      $\mathbf{g}_{pj} = \mathcal{G}(\mathbf{k}_p, \mathbf{k}_j)$ 
6:   end for
7:    $k_M = \sum \mathbf{w}_j k_j, \mathbf{w}_j = \mathbf{g}_{pj} / \sum \mathbf{g}_{pj}$ 
8:   return  $k_M$ 
9: end procedure

```

KVMerger also retains the KV states whose corresponding aggregated attention scores fall within the top-k range, including both attention sinks and heavy-hitters, which represent the most important and frequently accessed elements by LLMs. We assume that those key and value states are quite sensitive to merging and cannot participant in merging process to avoid information damage.

4.2 GAUSSIAN KERNEL WEIGHTED MERGING

Definition 4.2 (Weighted KV cache Merging, formal). *Given identified merging sets of key states and value states as $\mathcal{S}_k = \{k_i, k_{i+1}, \dots, k_p, \dots, k_{i+n}\}$ and $\mathcal{S}_v = \{v_i, v_{i+1}, \dots, v_p, \dots, v_{i+n}\}$, where k_p and v_p denote the pivotal key state and pivotal value state, respectively. Then, the weighted merging key states and value states can be defined as:*

$$k_p = w_p k_p + \sum_{k_i \in \mathcal{S}_k, i \neq p} w_i k_i, \quad v_p = w_p v_p + \sum_{v_i \in \mathcal{S}_v, i \neq p} w_i v_i \quad (5)$$

where w_p and w_i denote the weight assigned to the pivotal state and the remaining states in the merging set.

We define the weighted merging function for KV cache merging in Definition 4.2, which follows the many-to-one merging definitions from Wan et al. (2024). Note that the merging function is a critical determinant of performance in many-to-one merging scenario. Two principal design factors directly influence merging efficacy. The first factor is the selection of the pivotal state, to which all other states are merged. The second factor involves the assignment of weights to each state, with the pivot state having the largest weight to preserve the information.

We start from the most intuitive merging function via average weighted for each merging set. We evaluate the average weighted merging function on four tasks from LongBench: TREC, NarrativeQA, TriviaQA, and LCC. As highlighted in previous research (Xiao et al., 2024) (Zhang et al., 2023) (Wan et al., 2024), recent tokens play a crucial role in performance. Therefore, we exclude the most recent tokens from merging to maintain an average compression ratio of 50%, as discussed in Section 3.2. For simplicity, we select the pivotal token as the key state with the maximum index within each merging set. Additionally, we compare the average weighted merging function with the H2O algorithm to gain an initial perspective on the potential and performance differences between

the KV cache merging scheme and the eviction scheme. The evaluation results are shown in Figure 3(c). The results demonstrate that the average weighted merging scheme provides a robust baseline, affirming the efficacy of the current method for identifying merging sets. However, the average weighted merging function performs worse compared to H2O, suggesting that the merging process may introduce noise, leading to information distortion.

Gaussian Kernel Weights To eliminate the noise introduced by less informative key states via average weighted merging, we introduce Gaussian kernel weighted merging, which is expressed as:

$$\mathbf{g}_{pi} = \mathcal{G}(\mathbf{k}_p, \mathbf{k}_i) = \exp\left(-\frac{\|\mathbf{k}_p - \mathbf{k}_i\|^2}{2\sigma^2}\right), \quad \sigma = \frac{\sum_0^{|\mathbf{S}_k|} \mathbf{g}_{pi}}{\sqrt{2}|\mathbf{S}_k|}. \quad (6)$$

Gaussian kernel is able to assign greater weight to elements that are nearer to the pivotal state. This local weighting characteristic ensures that the merged result is significantly shaped by nearby states, maintaining local structure and minimizing the impact of distant, possibly noisy states. Then, the merging weights for key states and value states can be formalized as:

$$\mathbf{w}_i = \frac{\mathbf{g}_{pi}}{\sum_{j=0}^{|\mathbf{S}_k|} \mathbf{g}_{pj}}, \quad \mathbf{w}_p = \frac{1}{\sum_{j=0}^{|\mathbf{S}_k|} \mathbf{g}_{pj}}. \quad (7)$$

For merging value states, $\mathbf{w}_i$ and $\mathbf{w}_p$ should also multiple $|\mathbf{S}_v|$. This adjustment is necessitated by the need to accurately reflect the number of value states during the merging process, ensuring that the merged result accurately represents the contribution of each value state. As demonstrated in Definition 4.2, the weight assigned to each $\mathbf{k}_i$ and $\mathbf{v}_i$ is directly governed by the squared l_2 norm between the pivotal token and the remaining tokens. This indicates that if $\mathbf{k}_i$ is close to $\mathbf{k}_p$ in the Euclidean space, more weight will be assigned to $\mathbf{k}_i$ as Figure 3(d) illustrates. Therefore, the value of σ is crucial as it directly determine the weights. Specifically, if σ approaches 0 and $\|\mathbf{k}_p - \mathbf{k}_i\|^2$ is significantly different from 0, the weight assigned to $\mathbf{k}_i$ tends towards 0. Consequently, no additional information is preserved except for the pivotal token. We empirically define σ as the mean value of $\mathbf{g}_{pi}$ for all tokens within each merging set to avoid such situation.

Selection for Pivotal State As previously discussed, the selection for pivotal state within each merging set is crucial for performance. Here we follow previous token eviction methods that using aggregated attention score to select pivotal token as it indicates the importance of tokens, which can be expressed as:

$$\mathbf{k}_p = \underset{i \in \mathbf{S}_k}{\operatorname{argmax}} (\mathbf{a}_i), \quad \mathbf{a}_i = \sum_{i=0}^{|\mathbf{S}_k|} A[i, :] \quad (8)$$

Note that the index of pivotal token for value states within each merging set is the same as key states. The complete merging policy is described as Algorithm 2.

5 EXPERIMENT

5.1 EXPERIMENTAL SETTINGS

Models and Tasks We evaluate *KVMerger* using three models: Llama2-7B/13B-chat (Touvron et al., 2023b) and Mistral-7B-Instruct-v1.0(Jiang et al., 2023). Our evaluation focuses primarily on instruction-tuned models, as these are meticulously optimized for dialogue use cases and question-answering scenarios. The above three models are evaluated on two commonly used benchmarks for long-context scenario, that is, LongBench (Bai et al., 2024) and ZeroScrolls (Shaham et al., 2023). Both LongBench and ZeroScrolls include a variety of scenarios such as multi-document question answering, summarization, and dialogue generation, providing a comprehensive assessment of a model’s ability to handle long sequences of text while maintaining coherence and accuracy. Specifically, we use nine datasets in LongBench: 2WikiMQA, gov_report, NarrativeQA, passage_retrieval_en, MultifieldQA_en, TREC, multi_news, TriviaQA, qasper. We use seven datasets in ZeroScrolls: gov_report, SummScreenFD, QMSum, SQuALITY, Qasper, NarrativeQA, BookSumSort. Additionally, we also individually test our methods on RAG tasks with the Needle-in-a-Haystack test (Guerreiro et al., 2023). The performance of our method for LLMs on all the above tasks are also compared with existing eviction method H2O and merging method CaM.

Table 1: *KVMerger* for Llama2-7B/13B-chat and Mistral-7B-Instruct on LongBench datasets.

models	budget	method	2wikimqa	gov_report	narrativeqa	pr_en	multifieldqa_en	trec	multi_news	triviaqa	qasper	avg.
Llama2-7B-chat	100%	Full Cache	31.45	26.99	18.74	8.00	36.60	64.00	26.26	83.09	21.83	35.22
		H2O	29.96	24.86	17.48	7.00	33.58	63.50	26.00	82.51	21.04	34.00
		CaM	30.69	24.46	17.08	6.50	33.98	63.50	24.66	82.17	20.00	33.67
	50%	<i>KVMerger</i>	32.99	25.31	18.50	7.33	36.89	64.00	26.29	83.62	20.04	35.02
		H2O	30.57	24.48	17.85	7.00	32.17	63.00	25.37	80.89	20.04	33.49
		CaM	31.06	23.80	18.36	6.00	33.07	62.50	25.23	81.86	18.37	33.36
	35%	<i>KVMerger</i>	32.29	25.24	19.12	7.00	33.82	63.50	25.64	82.76	21.09	34.50
		Full Cache	13.21	27.59	14.42	15.25	27.44	68.50	26.69	87.42	17.15	33.07
		H2O	13.39	26.20	15.01	15.50	26.40	68.00	25.35	84.73	17.10	32.40
Llama2-13B-chat	100%	Full Cache	13.21	27.59	14.42	15.25	27.44	68.50	26.69	87.42	17.15	33.07
		H2O	13.39	26.20	15.01	15.50	26.40	68.00	25.35	84.73	17.10	32.40
		CaM	13.30	25.88	13.47	15.00	26.96	67.50	26.06	84.65	16.58	32.16
	50%	<i>KVMerger</i>	13.46	26.63	14.4	16.00	27.29	68.50	26.12	87.48	17.22	33.01
		H2O	12.26	25.52	13.14	14.50	25.75	67.50	25.59	83.53	16.35	31.57
		CaM	13.43	25.37	13.58	12.50	25.70	67.50	25.04	84.95	16.34	31.60
	35%	<i>KVMerger</i>	12.61	26.12	13.60	14.00	26.75	68.00	26.32	86.76	16.24	32.27
		Full Cache	31.47	26.55	21.96	25.00	39.50	61.00	26.44	83.89	30.12	38.44
		H2O	29.21	19.91	17.65	8.00	25.50	53.00	19.95	74.55	21.51	29.92
Mistral-7B-Instruct	100%	Full Cache	31.47	26.55	21.96	25.00	39.50	61.00	26.44	83.89	30.12	38.44
		H2O	29.21	19.91	17.65	8.00	25.50	53.00	19.95	74.55	21.51	29.92
		CaM	29.57	22.67	19.43	12.00	28.95	58.00	20.17	81.82	21.87	32.72
	50%	<i>KVMerger</i>	32.44	24.05	21.85	23.00	31.23	60.00	20.87	84.16	24.52	35.79
		H2O	12.30	5.16	3.64	0.62	11.95	37.50	18.99	17.08	14.05	13.48
		CaM	28.77	18.70	17.76	8.50	25.31	45.50	19.72	72.88	17.25	28.27
	35%	<i>KVMerger</i>	30.77	20.99	23.58	23.50	28.10	60.5	19.94	83.82	24.13	35.04
		Full Cache	31.47	26.55	21.96	25.00	39.50	61.00	26.44	83.89	30.12	38.44
		H2O	29.21	19.91	17.65	8.00	25.50	53.00	19.95	74.55	21.51	29.92

Implementation details We test *KVMerger* in two compression scenarios. The first one is 50% KV cache budget, where the proportion of recent tokens to be reserved is set to 0.17%, and the proportion of states not selected for the merging process in terms of aggregated attention scores is set to 0.12%. The remaining key states and value states participate in the merging process. The second compression scenario is 35% KV cache budget, where the proportion of recent tokens is set to 0.08%, and the proportion of states not selected for the merging process is set to 0.02%. The cosine similarity threshold for both two scenarios is set to 0.75. We conducted our experiments on a cluster with A100 40GB GPUs (4XA100 per node, 256GB DRAM, 15TB storage, 200Gbps interconnect), and a cluster with A100 80GB GPUs (2xA100 per node, 256GB DRAM, 100Gb Ethernet interconnect). The evaluation process for LongBench and ZeroScrolls follows THUDM (2024) and Lab (2024). The implementation of Needle-in-a-Haystack test follows Kamradt.

5.2 EXPERIMENTAL RESULTS ON LONG-CONTEXT TASKS

LongBench Results The evaluation results of nine selected LongBench datasets on Llama2-7B/13B-chat and Mistral-7B-Instruct-v1.0 are shown in Table 1. We compare the current KV cache compression methods, including H2O, CaM, with our proposed KV merging method *KVMerger* by preserving both 50% and 35% of contexts in the KV cache. Our results demonstrate that *KVMerger* consistently outperforms the other KV cache compression techniques across nearly all selected datasets from LongBench. Notably, the performance gaps between our algorithm and the full KV cache scenario for both Llama2-7B/13B-chat and Mistral-7B-Instruct-v1.0 are significantly smaller than the other KV compression methods. More importantly, our method achieves better evaluation results on several tasks compared to the full cache scenario, highlighting the efficacy and robustness of our approach on long-context tasks. Another interesting finding is that the latest value states merging method, CaM, does not perform well on long-context tasks. This may be attributed to the information loss results from eviction of key states, despite the merging of value states.

Mistral-7B-Instruct-v1.0 leverages the Grouped-Query-Attention (GQA) technique to optimize KV cache memory usage. In this approach, each key state corresponds to four query states. When applying the H2O method to each key state, rather than duplicating key and value states, we use a single attention map. This attention map is generated by averaging the values of four attention maps formed by the four query states, which determines the states to be evicted. For the *KVMerger* method, we also utilize this singular attention map to select pivotal states, ensuring a fair comparison. Our results indicate a significant performance drop for Mistral-7B-Instruct-v1.0 when using the H2O method. Conversely, *KVMerger* demonstrates the smallest performance decline under both 35% and 50% KV cache budgets, highlighting its efficiency on GQA.

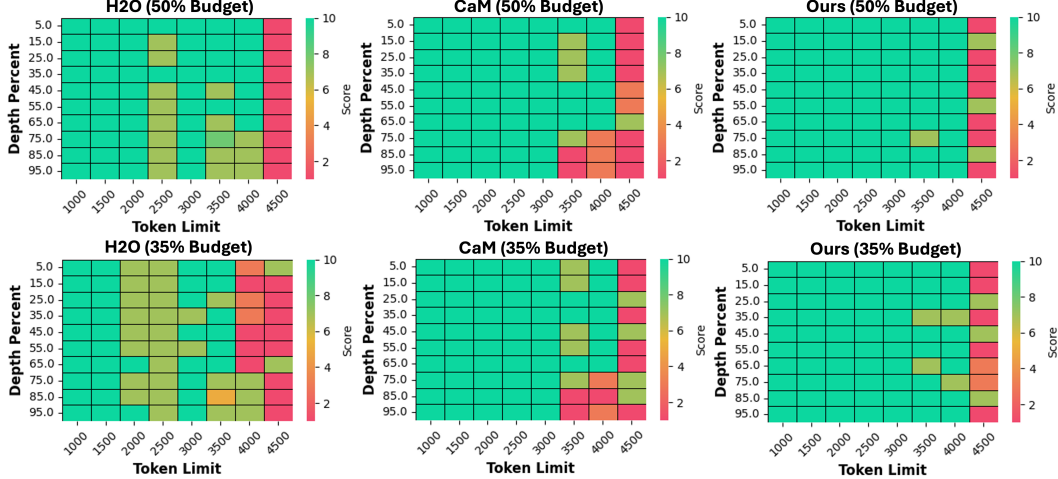


Figure 5: The visualization of needle-in-a-haystack test on Llama2-7B-chat with different KV cache compression methods. The x-axis represents the length of contexts, and the y-axis represents the document depth where the needle is inserted.

ZeroScrolls Results We also evaluate Llama2-7B-chat on ZeroScrolls datasets using different KV cache compression techniques, as shown in Table 2. The ZeroScrolls datasets are characterized by an average sample length of approximately 4300 words per topic, closely matching the maximum window size of pre-trained Llama2-7B-chat models. This alignment indicates that the datasets are well-suited for these models, ensuring effective processing and analysis without the risk of truncating important information. Table 2 demonstrates that our proposed KV cache merging method effectively restores the performance of the Llama2-7B-chat model across all selected ZeroScrolls datasets under both 35% and 50% cache budgets. This suggests that *KVMerger* not only mitigates performance degradation but also optimizes the model’s handling of extensive data sequences that approach the model’s maximum context window, contributing to more robust outcomes.

Table 2: *KVMerger* for Llama2-7B-chat on selected **ZeroScrolls** datasets

cache budget	Method	gov_report	SummScreenFD	QMSum	SQuALITY	Qasper	NarrativeQA	BookSumSort	avg.
100%	Full Cache	17.40	14.10	15.20	19.50	22.50	15.40	3.00	15.30
50%	H2O	15.40	13.20	14.30	18.30	20.50	15.00	3.80	14.36
	CaM	15.60	13.10	13.70	18.50	20.10	15.30	3.40	14.24
	<i>KVMerger</i>	17.70	13.80	15.10	19.10	22.50	15.20	3.10	15.21
35%	H2O	14.80	11.60	14.20	17.80	17.70	14.70	3.60	13.49
	CaM	15.30	11.70	13.90	18.30	17.10	14.50	3.30	13.44
	<i>KVMerger</i>	16.60	13.80	15.40	18.60	20.40	15.40	3.70	14.84

Needle In A Haystack Results We also conduct a detailed comparison of *KVMerger* with other KV cache compression techniques on retrieval tasks using the needle-in-a-haystack test. This test involves placing a random fact or statement in the middle of a long context window and assessing the model’s ability to retrieve this statement across varying document depths and context lengths to measure performance. Specifically, we test Llama2-7B-chat on document depths ranging from 5% to 95% and context lengths ranging from 1000 to 4500 tokens under both 35% and 50% cache budgets. The corresponding results are illustrated as Figure 5. Our findings indicate that both CaM and our merging algorithm outperform the eviction method H2O. However, our proposed method achieves the highest retrieval performance, consistently delivering high scores across various context lengths and depth percentages. Notably, even when the context length exceeds the pre-trained context length of the Llama2-7B-chat model, our method maintains high scores at specific depth percentages.

5.3 ABLATION STUDY

Choice of σ in Gaussian Kernel Weights In section 4.2, we set σ as the mean of Gaussian kernel weights within each merging set. This definition is based on the empirical observations by testing different values of σ . Specifically, we apply our proposed KV merging method on Llama2-7B-chat model with various pre-defined σ values under 50% cache budget. The experiments results on

selected datasets from LongBench are shown as Table 3. We can know that when σ equals to 5, the proposed KV cache merging method optimally recovers the model’s performance under a 50% cache budget. We then computed σ as expressed in Equation 4 for each merging set at different layers and found that the average value of computed σ for most layers fluctuates around 5, which aligns with the experiment results in Table 3.

Table 3: *KVMerger* with different σ values under 50% cache budget.

σ	2wikimqa	gov_report	narrativeqa	pr_en	multifieldqa_en	avg.
0.5	29.45	24.11	18.82	6.00	35.56	22.79
1	31.48	25.52	18.98	6.25	36.59	23.76
2	28.65	25.16	18.64	4.17	36.79	22.68
3	30.84	25.19	18.51	4.67	37.48	23.34
4	31.59	25.65	18.09	5.83	36.25	23.48
5	32.99	25.31	18.50	7.33	36.89	24.20
6	31.69	25.39	18.45	7.83	35.82	23.84

Choice of Pivotal State in Gaussian Kernel Weighted Merging As mentioned in Section 4.2, the selection of pivotal state for each merging set is directly related to the performance of *KVMerger*. The inappropriate selection of pivotal states will result in the severe information distortion and even much worse information loss than eviction-based compression algorithms. To show the significance of defining the pivotal state as the state with the biggest aggregated attention scores, we compare it with randomly selecting pivotal state within each merging set by using Llama2-7B-chat model with 50% cache budget. The comparison is shown in Table 4, from which we can see that randomly selecting pivotal states are detrimental to LLMs’ performance on long-context tasks.

Table 4: *KVMerger* with different methods of pivotal states selection.

Pivotal State	2wikimqa	gov_report	narrativeqa	pr_en	multifieldqa_en	avg.
Ours	32.99	25.31	18.50	7.33	36.89	24.20
Random	30.01	24.07	17.72	6.50	33.30	22.12

6 CONCLUSION AND FUTURE WORK

In this paper, we propose *KVMerger*, a dynamic KV cache merging method inspired by the observation that key states exhibit high and persistent similarity within each sequence, allowing for layer-wise KV cache compression. We initially abstract the merging set identification problem as a constrained clustering problem and introduce a variant of the AHC algorithm to identify merging sets based on cosine similarities between key states. Furthermore, we implement a Gaussian Kernel weighted merging method to merge key and value states within each merging set. Compared to other KV cache eviction and merging methods, our approach achieves superior results on the LongBench datasets under the same cache budget. Additionally, our method effectively recovers the model’s long-context retrieval capabilities, as demonstrated by the needle-in-a-haystack tests.

Future work can explore several avenues to enhance and extend our proposed method. First, investigating the impact of different clustering algorithms and similarity measurements could provide insights into further optimizing the merging sets. Second, applying our method to other LLMs including long-context fine-tuned models and datasets would help assess its generalizability and robustness. Third, exploring hybrid approaches that combine cache merging with other memory management techniques might yield even more efficient solutions for long-context retrieval tasks.

ACKNOWLEDGMENTS

This work used Delta system at the National Center for Supercomputing Applications through allocation CIS240055 from the Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support (ACCESS) program. ACCESS (Boerner et al., 2023) is an advanced computing and data resource program supported by the U.S. National Science Foundation (NSF) under the Office of Advanced Cyberinfrastructure awards #2138259, #2138286, #2138307, #2137603 and #2138296. The Delta advanced computing resource is a joint effort of the University of Illinois Urbana-Champaign and the National Center for Supercomputing Applications, and it is supported by the National Science Foundation (award OAC 2005572) and the State of Illinois. The work also used the Illinois Campus Cluster and NCSA NFI Hydro cluster, which are supported by the University of Illinois Urbana-Champaign and the University of Illinois System.

REFERENCES

- Anthropic. Claude. <https://www.anthropic.com/claude>, 2023. Language model developed by Anthropic.
- Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. Longbench: A bilingual, multitask benchmark for long context understanding, 2024. URL <https://arxiv.org/abs/2308.14508>.
- Timothy J. Boerner, Stephen Deems, Thomas R. Furlani, Shelley L. Knuth, and John Towns. ACCESS: Advancing Innovation: NSF’s Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support. In *Practice and Experience in Advanced Research Computing (PEARC’23)*, 2023.
- Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, Christoph Feichtenhofer, and Judy Hoffman. Token merging: Your vit but faster, 2023. URL <https://arxiv.org/abs/2210.09461>.
- Jincheng Dai, Zhuowei Huang, Haiyun Jiang, Chen Chen, Deng Cai, Wei Bi, and Shuming Shi. Corm: Cache optimization with recent message for large language model inference, 2024. URL <https://arxiv.org/abs/2404.15949>.
- Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. Model tells you what to discard: Adaptive kv cache compression for llms, 2024. URL <https://arxiv.org/abs/2310.01801>.
- etc. Gemini Team, Petko Georgiev. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context, 2024. URL <https://arxiv.org/abs/2403.05530>.
- Nuno M. Guerreiro, Elena Voita, and André F. T. Martins. Looking for a needle in a haystack: A comprehensive study of hallucinations in neural machine translation, 2023. URL <https://arxiv.org/abs/2208.05309>.
- Yefei He, Luoming Zhang, Weijia Wu, Jing Liu, Hong Zhou, and Bohan Zhuang. Zipcache: Accurate and efficient kv cache quantization with salient token identification, 2024. URL <https://arxiv.org/abs/2405.14256>.
- Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. Kvquant: Towards 10 million context length LLM inference with KV cache quantization. *CoRR*, abs/2401.18079, 2024.
- Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Léo Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mistral 7b, 2023. URL <https://arxiv.org/abs/2310.06825>.
- George Kamradt. Llmtest: Needle in a haystack. https://github.com/gkamradt/LLMTest_NeedleInAHaystack. Accessed: 2024-07-08.
- Hao Kang, Qingru Zhang, Souvik Kundu, Geonhwa Jeong, Zaoxing Liu, Tushar Krishna, and Tuo Zhao. Gear: An efficient kv cache compression recipe for near-lossless generative inference of llm, 2024. URL <https://arxiv.org/abs/2403.05527>.
- Minchul Kim, Shangqian Gao, Yen-Chang Hsu, Yilin Shen, and Hongxia Jin. Token fusion: Bridging the gap between token pruning and token merging, 2023. URL <https://arxiv.org/abs/2312.01026>.
- TAU NLP Lab. Zeroscrolls: Zero-shot summarization and reasoning language system. https://github.com/tau-nlp/zero_scroll, 2024. Accessed: 2024-07-02.
- Akide Liu, Jing Liu, Zizheng Pan, Yefei He, Gholamreza Haffari, and Bohan Zhuang. Minicache: Kv cache compression in depth dimension for large language models, 2024. URL <https://arxiv.org/abs/2405.14366>.

-
- Zechun Liu, Barlas Oguz, Changsheng Zhao, Ernie Chang, Pierre Stock, Yashar Mehdad, Yangyang Shi, Raghuraman Krishnamoorthi, and Vikas Chandra. LLM-QAT: data-free quantization aware training for large language models. *CoRR*, abs/2305.17888, 2023a.
- Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhao Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. Scissorhands: Exploiting the persistence of importance hypothesis for llm kv cache compression at test time, 2023b. URL <https://arxiv.org/abs/2305.17118>.
- OpenAI. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- Alexander Peysakhovich and Adam Lerer. Attention sorting combats recency bias in long context language models, 2023. URL <https://arxiv.org/abs/2310.01427>.
- Siyu Ren and Kenny Q. Zhu. On the efficacy of eviction policy for key-value constrained generative language model inference, 2024. URL <https://arxiv.org/abs/2402.06262>.
- Uri Shaham, Maor Ivgi, Avia Efrat, Jonathan Berant, and Omer Levy. Zeroscrolls: A zero-shot benchmark for long text understanding, 2023. URL <https://arxiv.org/abs/2305.14196>.
- Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. Flexgen: High-throughput generative inference of large language models with a single GPU. In *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, volume 202 of *Proceedings of Machine Learning Research*, pp. 31094–31116. PMLR, 2023.
- Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding, 2023. URL <https://arxiv.org/abs/2104.09864>.
- Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context llm inference, 2024. URL <https://arxiv.org/abs/2406.10774>.
- THUDM. Longbench. <https://github.com/THUDM/LongBench>, 2024. Accessed: 2024-07-02.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023a.
- Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023b.
- Zhongwei Wan, Xinjian Wu, Yu Zhang, Yi Xin, Chaofan Tao, Zhihong Zhu, Xin Wang, Siqi Luo, Jing Xiong, and Mi Zhang. D2o: Dynamic discriminative operations for efficient generative inference of large language models, 2024. URL <https://arxiv.org/abs/2406.13035>.
- Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks, 2024. URL <https://arxiv.org/abs/2309.17453>.
- June Yong Yang, Byeongwook Kim, Jeongin Bae, Beomseok Kwon, Gunho Park, Eunho Yang, Se Jung Kwon, and Dongsoo Lee. No token left behind: Reliable kv cache compression via importance-aware mixed precision quantization, 2024. URL <https://arxiv.org/abs/2402.18096>.
- Hao Yu, Zelan Yang, Shen Li, Yong Li, and Jianxin Wu. Effectively compress kv heads for llm, 2024a. URL <https://arxiv.org/abs/2406.07056>.
- Zhongzhi Yu, Zheng Wang, Yonggan Fu, Huihong Shi, Khalid Shaikh, and Yingyan Celine Lin. Unveiling and harnessing hidden attention sinks: Enhancing large language models without training through attention calibration, 2024b. URL <https://arxiv.org/abs/2406.15765>.

-
- Wang Zeng, Sheng Jin, Wentao Liu, Chen Qian, Ping Luo, Wanli Ouyang, and Xiaogang Wang. Not all tokens are equal: Human-centric visual analysis via token clustering transformer, 2022. URL <https://arxiv.org/abs/2204.08680>.
- Liang Zhang, Anwen Hu, Haiyang Xu, Ming Yan, Yichen Xu, Qin Jin, Ji Zhang, and Fei Huang. Tinychart: Efficient chart understanding with visual token merging and program-of-thoughts learning, 2024a. URL <https://arxiv.org/abs/2404.16635>.
- Tianyi Zhang, Jonah Yi, Zhaozhao Xu, and Anshumali Shrivastava. Kv cache is 1 bit per channel: Efficient large language model inference with coupled quantization, 2024b. URL <https://arxiv.org/abs/2405.03917>.
- Yuxin Zhang, Yuxuan Du, Gen Luo, Yunshan Zhong, Zhenyu Zhang, Shiwei Liu, and Rongrong Ji. Cam: Cache merging for memory-efficient llms inference. In *Forty-first International Conference on Machine Learning*.
- Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H₂o: Heavy-hitter oracle for efficient generative inference of large language models, 2023. URL <https://arxiv.org/abs/2306.14048>.
- Zhenyu Zhang, Shiwei Liu, Runjin Chen, Bhavya Kailkhura, Beidi Chen, and Atlas Wang. Q-hitter: A better token oracle for efficient llm inference via sparse-quantized kv cache. *Proceedings of Machine Learning and Systems*, 6:381–394, 2024c.
- Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhao Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. Kivi : Plug-and-play 2bit kv cache quantization with streaming asymmetric quantization. 2023. doi: 10.13140/RG.2.2.28167.37282. URL <https://rgdoi.net/10.13140/RG.2.2.28167.37282>.

A APPENDIX

Lemma 3.1 (Formal version of Lemma 3.1). *Consider two vectors $\mathbf{k}_m, \mathbf{k}_n \in \mathbb{R}^{1 \times d}$. If their cosine similarity is 1, then the cosine similarity of any 1×2 vectors, $\mathbf{k}_{m,j} = [k_{m,j}, k_{m,2j+1}]^T$ and $\mathbf{k}_{n,j} = [k_{n,2j}, k_{n,2j+1}]^T$, formed by the $2j$ -th and $(2j+1)$ -th elements of $\mathbf{k}_m$ and $\mathbf{k}_n$, $0 \leq j \leq \frac{d-1}{2}$, is also equal to 1.*

Proof. Since

$$\text{similarity}(\mathbf{k}_m, \mathbf{k}_n) = 1, \quad (9)$$

$\mathbf{k}_m$ and $\mathbf{k}_n$ are collinear. Therefore,

$$\mathbf{k}_m = \alpha \mathbf{k}_n, \quad (10)$$

where α is a scalar. It means

$$k_{m,2j} = \alpha k_{n,2j}, \quad (11)$$

$$k_{m,2j+1} = \alpha k_{n,2j+1}. \quad (12)$$

So,

$$[k_{m,2j}, k_{m,2j+1}]^T = \alpha [k_{n,2j}, k_{n,2j+1}]^T. \quad (13)$$

As a result,

$$\text{similarity}(\mathbf{k}_{m,j}, \mathbf{k}_{n,j}) = 1 \quad (14)$$

Lemma 3.2 (Formal version of Lemma 3.2). *Consider integer j such that $0 \leq j \leq \frac{d-1}{2}$. Define the vectors $\mathbf{k}_{m,j}$ and $\mathbf{k}_{n,j}$ as $\mathbf{k}_{m,j} = [k_{m,2j}, k_{m,2j+1}]^T$ and $\mathbf{k}_{n,j} = [k_{n,2j}, k_{n,2j+1}]^T$, and define the vectors $\mathbf{k}'_{m,j}$ and $\mathbf{k}'_{n,j}$ as $\mathbf{k}'_{m,j} = \mathbf{k}_{m,j}/e^{im\theta_j}$ and $\mathbf{k}'_{n,j} = \mathbf{k}_{n,j}/e^{in\theta_j}$. If $\text{similarity}(\mathbf{k}_{m,j}, \mathbf{k}_{n,j}) = 1$, we have:*

$$\cos(m - n) < \frac{\langle \mathbf{k}'_{m,j}, \mathbf{k}'_{n,j} \rangle}{\|\mathbf{k}'_{m,j}\| \cdot \|\mathbf{k}'_{n,j}\|} \leq 1,$$

where $\langle \mathbf{k}'_{m,j}, \mathbf{k}'_{n,j} \rangle$ denotes the inner product of $\mathbf{k}'_{m,j}$ and $\mathbf{k}'_{n,j}$, and $\|\mathbf{k}'_{m,j}\|$ and $\|\mathbf{k}'_{n,j}\|$ denote the norms of $\mathbf{k}'_{m,j}$ and $\mathbf{k}'_{n,j}$, respectively.

Proof. Since j is an integer obeying $0 \leq j \leq \frac{d-1}{2}$, so

$$-1 < \frac{-2j}{d} \leq 0. \quad (15)$$

And b is set to be 10000 by default Su et al. (2023). Therefore,

$$0 < b^{\frac{-2j}{d}} \leq 1, \quad (16)$$

which means

$$0 < \theta_j \leq 1. \quad (17)$$

Now, focus on the similarity between $\mathbf{k}_{m,j}$ and $\mathbf{k}_{n,j}$, and

$$\frac{\langle \mathbf{k}_{m,j}, \mathbf{k}_{n,j} \rangle}{\|\mathbf{k}_{m,j}\| \cdot \|\mathbf{k}_{n,j}\|} = \frac{\langle \mathbf{k}'_{m,j} e^{im\theta_j}, \mathbf{k}'_{n,j} e^{in\theta_j} \rangle}{\|\mathbf{k}'_{m,j} e^{im\theta_j}\| \cdot \|\mathbf{k}'_{n,j} e^{in\theta_j}\|}. \quad (18)$$

It is easy to derive that

$$\|\mathbf{k}'_{m,j} e^{im\theta_j}\| = \|\mathbf{k}'_{m,j}\|, \quad (19)$$

$$\|\mathbf{k}'_{n,j} e^{in\theta_j}\| = \|\mathbf{k}'_{n,j}\|, \quad (20)$$

since the exponential terms do not change the vectors' magnitude.

Then, substitute the complex forms of $\mathbf{k}'_{m,j}$ and $\mathbf{k}'_{n,j}$, $\mathbf{k}'_{m,j} = k'_{m,2j} + ik'_{m,2j+1}$ and $\mathbf{k}'_{n,j} = k'_{n,2j} + ik'_{n,2j+1}$, respectively, back into $\langle \mathbf{k}'_{m,j} e^{im\theta_j}, \mathbf{k}'_{n,j} e^{in\theta_j} \rangle$ and obtain

$$\langle \mathbf{k}'_{m,j} e^{im\theta_j}, \mathbf{k}'_{n,j} e^{in\theta_j} \rangle = \langle (k'_{m,2j} + ik'_{m,2j+1}) e^{im\theta_j}, (k'_{n,2j} + ik'_{n,2j+1}) e^{in\theta_j} \rangle. \quad (21)$$

From Euler equation, Equation 21 can be further expanded as

$$\begin{aligned} & \langle \mathbf{k}'_{m,j} e^{im\theta_j}, \mathbf{k}'_{n,j} e^{in\theta_j} \rangle \\ &= k'_{m,2j} k'_{n,2j} \cos(m\theta_j) \cos(n\theta_j) + k'_{m,2j+1} k'_{n,2j+1} \sin(m\theta_j) \sin(n\theta_j) \\ & \quad - k'_{m,2j} k'_{n,2j+1} \cos(m\theta_j) \sin(n\theta_j) - k'_{m,2j+1} k'_{n,2j} \sin(m\theta_j) \cos(n\theta_j) \\ & \quad + k'_{m,2j} k'_{n,2j} \sin(m\theta_j) \sin(n\theta_j) + k'_{m,2j+1} k'_{n,2j+1} \cos(m\theta_j) \cos(n\theta_j) \\ & \quad + k'_{m,2j} k'_{n,2j+1} \sin(m\theta_j) \cos(n\theta_j) + k'_{m,2j+1} k'_{n,2j} \cos(m\theta_j) \sin(n\theta_j) \\ &= k'_{m,2j} k'_{n,2j} \cos[(m-n)\theta_j] + k'_{m,2j+1} k'_{n,2j+1} \cos[(m-n)\theta_j] \\ & \quad + k'_{m,2j} k'_{n,2j+1} \sin[(m-n)\theta_j] + k'_{m,2j+1} k'_{n,2j} \sin[(m-n)\theta_j]. \end{aligned} \quad (22)$$

Substitute Equation 22 back into Equation 18, and

$$\begin{aligned} \frac{\langle \mathbf{k}_{m,j}, \mathbf{k}_{n,j} \rangle}{\|\mathbf{k}_{m,j}\| \cdot \|\mathbf{k}_{n,j}\|} &= \frac{k'_{m,2j} k'_{n,2j} + k'_{m,2j+1} k'_{n,2j+1} \cos[(m-n)\theta_j]}{\|\mathbf{k}'_{m,j}\| \cdot \|\mathbf{k}'_{n,j}\|} \\ & \quad + \frac{k'_{m,2j} k'_{n,2j+1} - k'_{m,2j+1} k'_{n,2j} \sin[(m-n)\theta_j]}{\|\mathbf{k}'_{m,j}\| \cdot \|\mathbf{k}'_{n,j}\|} \\ &= \frac{\mathbf{k}'_{m,j} \cdot \mathbf{k}'_{n,j}}{\|\mathbf{k}'_{m,j}\| \cdot \|\mathbf{k}'_{n,j}\|} \cos[(m-n)\theta_j] + \frac{\mathbf{k}'_{m,j} \times \mathbf{k}'_{n,j}}{\|\mathbf{k}'_{m,j}\| \cdot \|\mathbf{k}'_{n,j}\|} \sin[(m-n)\theta_j]. \end{aligned} \quad (23)$$

Let ϕ be angle between $\mathbf{k}'_{m,j}$ and $\mathbf{k}'_{n,j}$, then Equation 23 can be rewrite as

$$\begin{aligned} \frac{\langle \mathbf{k}_{m,j}, \mathbf{k}_{n,j} \rangle}{\|\mathbf{k}_{m,j}\| \cdot \|\mathbf{k}_{n,j}\|} &= \cos \phi \cos[(m-n)\theta_j] + \sin \phi \sin[(m-n)\theta_j] \\ &= \cos[\phi - (m-n)\theta_j]. \end{aligned} \quad (24)$$

Since the similarity between $\mathbf{k}_{m,j}$ and $\mathbf{k}_{n,j}$ nearly equals 1 as we assumed, it can be obtained that

$$\phi = (m - n) \theta_j. \quad (25)$$

From Equation 17,

$$0 < \phi \leq m - n, \quad \text{if } m > n, \quad (26)$$

$$m - n \leq \phi < 0, \quad \text{if } m < n. \quad (27)$$

As a result,

$$\cos(m - n) < \frac{\langle \mathbf{k}'_{m,j}, \mathbf{k}'_{n,j} \rangle}{\|\mathbf{k}'_{m,j}\| \cdot \|\mathbf{k}'_{n,j}\|} \leq 1. \quad (28)$$